

Backpropagation: The Heart of Neural Learning

From Chain Rule to Multi-Layer Networks

MAI 604: Deep Learning

Department of Computational Sciences

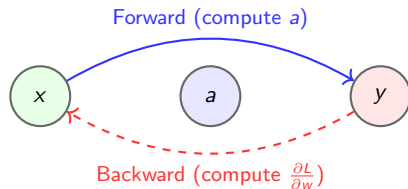
February 27, 2026

Lecture Outline

- 1 The Intuition
- 2 The Chain Rule
- 3 1-Node Network
- 4 Multi-Layer Network
- 5 Matrix Notation
- 6 Network Initialization

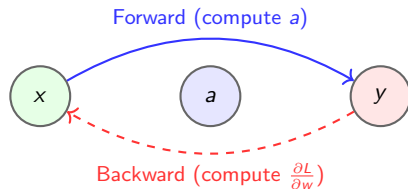
Optimization via Gradient Descent

- **Goal:** Calculate the gradient of the **Loss Function** $L(w)$.



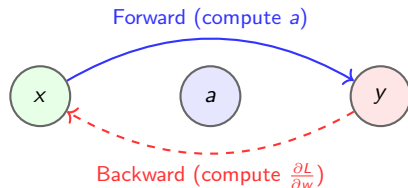
Optimization via Gradient Descent

- **Goal:** Calculate the gradient of the **Loss Function** $L(w)$.
- **Problem:** Deep networks are highly complicated.



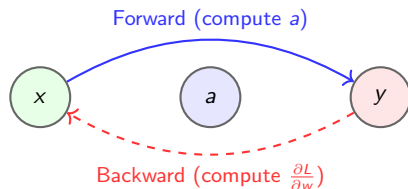
Optimization via Gradient Descent

- **Goal:** Calculate the gradient of the **Loss Function** $L(w)$.
- **Problem:** Deep networks are highly complicated.
- **Solution:** Backpropagation is an efficient way to calculate the partial derivatives.



Optimization via Gradient Descent

- **Goal:** Calculate the gradient of the **Loss Function** $L(w)$.
- **Problem:** Deep networks are highly complicated.
- **Solution:** Backpropagation is an efficient way to calculate the partial derivatives.



The Two-Phase Process

- 1 **Forward Pass:** Compute pre-activations (z), activations (a), and Loss.
- 2 **Backward Pass:** Propagate error (δ) from right to left.

The Engine: Basic Chain Rule

The Chain Rule Principle

If variable y depends on u , and u depends on x , then:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \cdot \frac{\partial u}{\partial x}$$

The Engine: Basic Chain Rule

The Chain Rule Principle

If variable y depends on u , and u depends on x , then:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \cdot \frac{\partial u}{\partial x}$$

Example: Let $y = \ln(x^2)$. Decompose this into:

The Engine: Basic Chain Rule

The Chain Rule Principle

If variable y depends on u , and u depends on x , then:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \cdot \frac{\partial u}{\partial x}$$

Example: Let $y = \ln(x^2)$. Decompose this into:

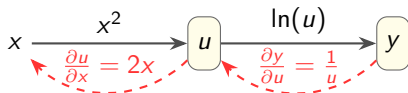
- **Inner:** $u = x^2 \implies \frac{\partial u}{\partial x} = 2x$
- **Outer:** $y = \ln(u) \implies \frac{\partial y}{\partial u} = \frac{1}{u}$

Applying the Rule

$$\frac{\partial y}{\partial x} = \frac{1}{u} \cdot 2x = \frac{1}{x^2} \cdot 2x = \frac{2}{x}$$

Visualizing the Chain Rule: $y = \ln(x^2)$

We can visualize the previous example as a flow of data.

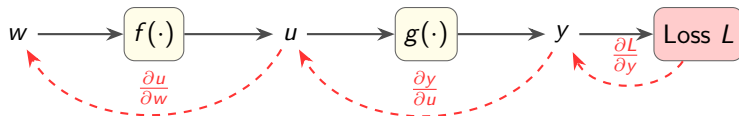


The Key Equation

$$\underbrace{\frac{\partial L}{\partial x}}_{\text{Downstream Gradient}} = \underbrace{\frac{\partial L}{\partial u}}_{\text{Upstream Gradient}} \cdot \underbrace{\frac{\partial u}{\partial x}}_{\text{Local Gradient}}$$

The Chain Rule as a Computational Graph

In Deep Learning, to find the gradient of loss L with respect to input w , we traverse the graph backwards.

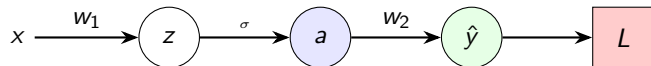


Total Gradient Calculation:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial u} \cdot \frac{\partial u}{\partial w}$$

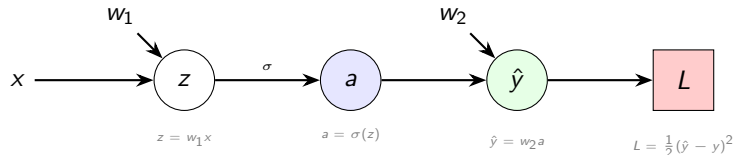
Case Study 1: Simple 1-Node Network

Simple Shallow Network:



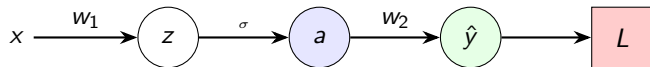
① Gradient for w_2 :

Different perspective (weights as inputs):



Case Study 1: Simple 1-Node Network

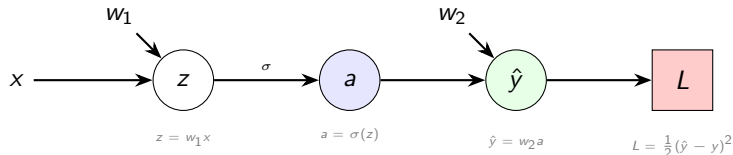
Simple Shallow Network:



① Gradient for w_2 :

$$\begin{aligned}\frac{\partial L}{\partial w_2} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2} \\ &= (\hat{y} - y) \cdot a \\ &= \delta \cdot c_1\end{aligned}$$

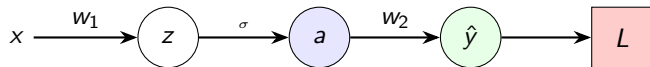
Different perspective (weights as inputs):



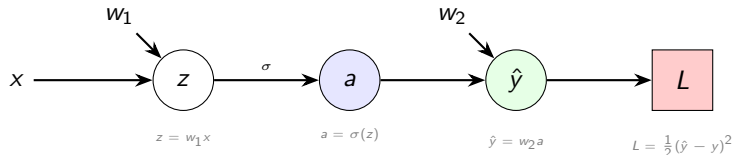
② Gradient for w_1 :

Case Study 1: Simple 1-Node Network

Simple Shallow Network:



Different perspective (weights as inputs):



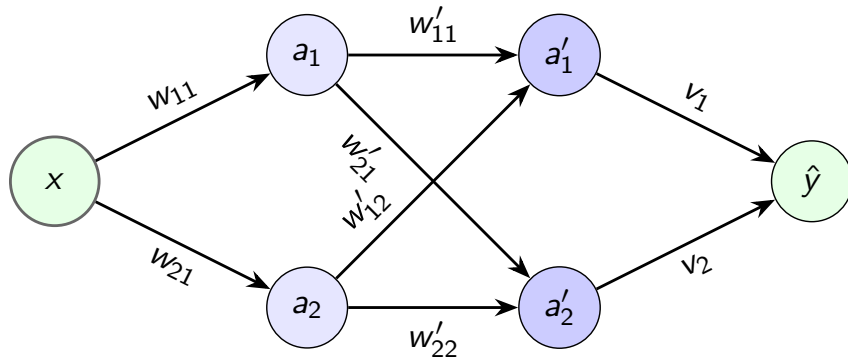
① Gradient for w_2 :

$$\begin{aligned}\frac{\partial L}{\partial w_2} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2} \\ &= (\hat{y} - y) \cdot a \\ &= \delta \cdot c_1\end{aligned}$$

② Gradient for w_1 :

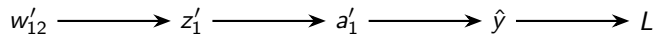
$$\begin{aligned}\frac{\partial L}{\partial w_1} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_1} \\ &= (\hat{y} - y) \cdot w_2 \cdot \sigma'(z) \cdot x \\ &= \delta \cdot c_2\end{aligned}$$

Case Study 2: 2 Hidden Layers, 2 Nodes



Backprop for Weight w'_{12}

The Path to Loss:



Backprop for Weight w'_{12}

The Path to Loss:

$$w'_{12} \longrightarrow z'_1 \longrightarrow a'_1 \longrightarrow \hat{y} \longrightarrow L$$

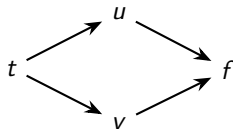
Applying the Chain Rule:

$$\begin{aligned} \frac{\partial L}{\partial w'_{12}} &= \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a'_1} \cdot \frac{\partial a'_1}{\partial z'_1} \cdot \frac{\partial z'_1}{\partial w'_{12}} \\ &= (\hat{y} - y) \cdot v_1 \cdot \sigma'(z'_1) \cdot a_2 \end{aligned}$$

The Multivariate Chain Rule

1. General Concept

If a variable t influences a function f through multiple intermediate paths (e.g., u and v):

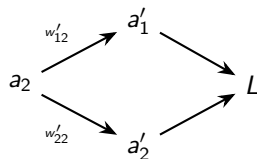


We **sum** the derivatives from each path:

$$\frac{\partial f}{\partial t} = \frac{\partial f}{\partial u} \cdot \frac{\partial u}{\partial t} + \frac{\partial f}{\partial v} \cdot \frac{\partial v}{\partial t}$$

2. Application to Subnetwork

Node a_2 influences the final Loss L through both nodes in the next layer (a'_1 and a'_2).



Summing the gradients from both paths:

$$\frac{\partial L}{\partial a_2} = \left(\frac{\partial L}{\partial a'_1} \cdot \frac{\partial a'_1}{\partial a_2} \right) + \left(\frac{\partial L}{\partial a'_2} \cdot \frac{\partial a'_2}{\partial a_2} \right)$$

Systematic Backprop: Step-by-Step for w_{21}

In practice, backpropagation calculates gradients node-by-node sequentially from output to input, caching intermediate errors to avoid massive equations.

Step 1: Accumulate Upstream Error at a_2

$$\begin{aligned}\frac{\partial L}{\partial a_2} &= \left(\frac{\partial L}{\partial a'_1} \cdot \frac{\partial a'_1}{\partial a_2} \right) + \left(\frac{\partial L}{\partial a'_2} \cdot \frac{\partial a'_2}{\partial a_2} \right) \\ &= \delta'_1 \cdot w'_{12} + \delta'_2 \cdot w'_{22}\end{aligned}$$

Step 2: Calculate Local Node Error (δ_2)

$$\begin{aligned}\frac{\partial L}{\partial z_2} &= \frac{\partial L}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \\ \delta_2 &= \frac{\partial L}{\partial a_2} \cdot \sigma'(z_2)\end{aligned}$$

Step 3: Compute Final Weight Gradient

$$\begin{aligned}\frac{\partial L}{\partial w_{21}} &= \frac{\partial L}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_{21}} \\ &= \delta_2 \cdot x\end{aligned}$$

Generalizing with Matrix Notation

1 Output Layer Error:

$$\delta^{(out)} = \nabla_{\hat{y}} L \odot \sigma'(\mathbf{z}^{(out)})$$

2 Hidden Layer Error:

$$\delta^{(l)} = \left((\mathbf{W}^{(l+1)})^T \delta^{(l+1)} \right) \odot \sigma'(\mathbf{z}^{(l)})$$

3 Weight Gradient Update:

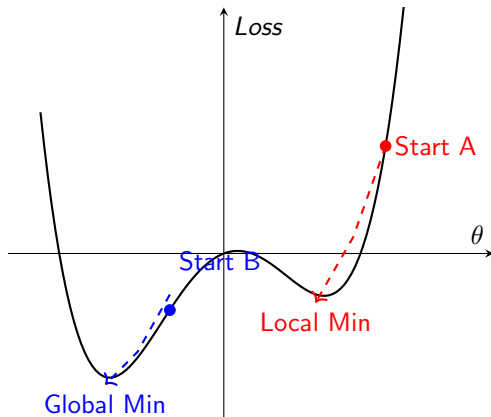
$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^T$$

Notation Note

The symbol $\odot$ denotes the **Hadamard Product** (element-wise multiplication).

Non-Convex Functions & Initialization

Real-world loss landscapes are often non-convex, containing multiple valleys (local minima).



The Initialization Problem:

- **Start A:** Gradient Descent flows into the nearest valley. It gets stuck in a **Local Minimum**.
- **Start B:** Starts in the basin of the **Global Minimum** and converges correctly.
- *Lesson:* Where you start matters!

Initialization Pitfalls in Neural Networks

Why can't we just set weights to zero or random large numbers?

1. Symmetry Breaking (All Zeros)

If $W = 0$, every neuron computes the exact same output.

- Backprop computes the same gradients for all weights.
- The network never learns complex features (stays linear).

2. Vanishing & Exploding Gradients

- **Too Small:** Activations decay to 0. Gradient vanishes.
- **Too Large:** Activations saturate (sigmoid) or explode (ReLU).

We need the variance of outputs to match the variance of inputs.

The Solution: He Initialization

For ReLU activations ($f(x) = \max(0, x)$), Kaiming He proposed a specific variance scaling to maintain signal strength.

He Initialization Formula

Initialize weights from a normal distribution with:

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{in}}}\right)$$

where n_{in} is the number of input connections to the neuron.

Why?

- It accounts for the fact that ReLU sets half the activations to zero.
- Keeps the variance of activations constant across layers, enabling deep networks to train.